

Exercise: MCP Anatomy

Background

The mCase MCP server ([@redmane/mcase-mcp-server](#)) gives Cursor the ability to query live mCase configuration data during a conversation. Understanding what tools it exposes and what data it returns is essential for building the capstone app.

You're going to use the [ai-agent-documentation](#) skill to understand the codebase rapidly — then use that knowledge to make design decisions.

Step 1 — Generate a Reference Doc (15 min)

The MCaseMCPTools codebase is open in Cursor. Invoke the [ai-agent-documentation](#) skill:

```
@ai-agent-documentation
```

```
Generate a minimal AI reference for this MCP server. Focus on:
```

1. What tools it exposes and what each one returns (include the data shapes, not just tool names)
2. The DatalistDefinition and FieldDefinition structures – what fields do they have and what do they represent?
3. What the templateProcessor filters out and why
4. What's missing from the current tools that a config browser app would want

While it generates, open [src/types.ts](#) to see the raw data shapes.

Step 2 — Understand the Data Model (5 min)

After the reference doc is generated, answer these questions:

Tools:

- Which tool returns a summary list of all datalists?
- Which tool returns the full definition of one datalist (with fields)?
- Which tool returns relationships between datalists?

Data shapes:

- What properties does a [FieldDefinition](#) have?
- What does [addType](#) represent? What values might it have?
- What is the [source](#) property on a FieldDefinition? What does it point to?
- What is the [values](#) array on a FieldDefinition?

Filtering:

- What kinds of fields does the `templateProcessor` strip out before returning data?
- Why might this be useful for a config browser?

Step 3 — Product Discovery (5 min)

Based on what the tools currently return, answer:

What can your capstone app do well with the current tools?

(List 2-3 things)

What is missing from the current tools that would make the app better?

(List 1-2 gaps)

If you were going to add one tool to the MCP server, what would it be?

Write a one-sentence description: `tool_name` — [what it accepts, what it returns]

Reference: MCP Tool Summary

Tool	Returns
<code>schema</code>	<code>DatalistSummary[]</code> — Id, Label, SystemName, FieldCount
<code>all_datalists</code>	Same as <code>schema</code>
<code>datalist_by_name</code>	<code>DatalistDefinition</code> — full detail including <code>Fields[]</code> , <code>Children[]</code>
<code>datalist_by_id</code>	Same as <code>datalist_by_name</code> using numeric Id
<code>all_relationships</code>	<code>Relationship[]</code> — <code>TargetDatalist</code> , <code>TargetField</code> , <code>SourceDatalist</code> , <code>SourceField</code> , <code>RelationshipType</code>
<code>greet</code>	Smoke test only — ignore

Reference: FieldDefinition Shape

```
{
  fieldId: number
  label: string           // Human-readable name
  systemName: string      // API/code name
  addType: string         // Field type: "Text", "Date", "Picklist", "Dynamic",
etc.
  required: boolean
  hidden: boolean
  isMirror: boolean
  values: string[]        // Picklist options (empty for non-picklist fields)
  source: number          // For Dynamic fields: ID of the linked datalist
  description: string     // Help text (HTML stripped)
}
```